# SKILL WEEK 7

## Task 1:

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
      pid_t
pid;
int status;

    pid = fork();
if (pid < 0) {
perror("fork failed");          return
1;
    }

    if (pid == 0) {

        // Child process          printf("Child process
started.\n");          printf("Child
PID: %d\n", getpid());

        sleep(3);

        printf("Child process completed.\n");

exit(0);

    } else {

        // Parent process          printf("Parent process
waiting for child...\n");

        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {              printf("Child exited
normally.\n");              printf("Child exit status: %d\n",
WEXITSTATUS(status));
        }
```

```
        printf("Parent process completed.\n");
    }

    return 0;
}
```

```
charith@DESKTOP-RBQQQSS:~$ nano s7.1.c
charith@DESKTOP-RBQQQSS:~$ gcc s7.1.c -o s7.1
charith@DESKTOP-RBQQQSS:~$ ./s7.1
Parent process waiting for child...
Child process started.
Child PID: 1764
Child process completed.
Child exited normally.
Child exit status: 0
Parent process completed.
```

## Task 2:

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main() {

    char *path = getenv("PATH");

    if (path == NULL) {         printf("PATH
variable not found.\n");         return 1;
    }

    printf("PATH:\n%s\n\n", path);

    char *copy = strdup(path);
char *directory = strtok(copy, ":");

    char filename[100];

    printf("Enter executable name to search: ");
scanf("%99s", filename);

    while (directory != NULL) {

        char fullpath[500];
```

```
        snprintf(fullpath, sizeof(fullpath),
                "%s/%s", directory, filename);

        if (access(fullpath, X_OK) == 0) {
printf("Executable found: %s\n", fullpath);
            free(copy);
return 0;
        }

        directory = strtok(NULL, ":");
    }

    printf("Executable not found in PATH.\n");

    free(copy);

    return 0;
}
```

Output:

